

GUIÓN DE DEMOSTRACIONES

Las seis demostraciones

Paso a paso, con texto hablado y qué hacer si falla



52 de los 90 minutos del webinar
Documento del demostrador

Contenido

Cómo se reparte la voz	4
Principios de demostración en webinar	4
Tabla de control	5
DEMO 1 • La skill en tres minutos	6
Objetivo	6
Estado inicial	6
Entrada	6
Guion	6
Señal de éxito	8
Cuando falla	8
DEMO 2 • Anatomía de un SKILL.md	9
Objetivo	9
Estado inicial	9
Entrada	9
Guion	9
Señal de éxito	10
Cuando falla	10
DEMO 3 • La descripción, A/B en vivo	11
Objetivo	11
Por qué es de alto riesgo	11
Estado inicial	11
Entrada	11
Guion	11
Señal de éxito	13
Cuando falla	13
DEMO 4 • El destilador	14
Objetivo	14
Por qué es de alto riesgo	14
Estado inicial	14
Entrada	14
Guion	14

Señal de éxito	16
Cuando falla	16

DEMO 5 · Multiarchivo y script

Objetivo	17
Estado inicial	17
Entrada	17
Guion	17
Señal de éxito	18
Cuando falla	18

DEMO 6 · La skill que no funciona

Objetivo	19
Estado inicial	19
Entrada	19
Guion	19
Señal de éxito	20
Cuando falla	20

Apéndice · Si van retrasados

Apéndice · Frases de cierre de cada demo

Las seis demos del webinar, al detalle. Todo lo que aparece en cita > es texto que puedes decir tal cual. Lo que está en bloque de código es lo que escribes en la terminal compartida.

Este documento es tuyo, del demostrador. Los cinco bloques hablados que van entre demostraciones son de Erick y están en el **Playbook**; no los necesitas para operar, pero conviene saber con qué te va a entregar la pantalla y con qué la retoma.

Cómo se reparte la voz

Cada demo empieza con un bloque **Entrada** y termina con un bloque **Salida**. Son las dos señales acordadas y no se improvisan.

	QUÉ ES	QUIÉN
Entrada	La frase con la que Erick te entrega la pantalla	Erick
Salida	La frase de cierre de la demo. Después de decirla, te callas	Tú

No agradezcas el cambio de turno. Cada "gracias Erick" cuesta ocho segundos y algo de ritmo, y en noventa minutos eso se acumula.

Si te pierdes: "Erick, ¿le seguimos con el mecanismo?" — frase neutral que suena planeada y le devuelve el control.

Si Erick dice "antes de que sigas con eso, una cosa rápida": corta donde estés y devuelve la pantalla. Significa que el reloj se salió de control. No se discute en vivo.

Modelo A o modelo B

Según lo que acuerden en el ensayo:

- **Modelo A** · tú manejas teclado y voz. Es lo que este guion supone.
- **Modelo B** · Erick narra y tú solo ejecutas. En ese caso, las citas de este documento las dice él, y tú reportas resultados en corto: "ya corrió", "aquí está el diff". Acuerden en el ensayo la señal de "ya puedes seguir".

Principios de demostración en webinar

Un webinar no es un taller. Nadie puede preguntarte a media demo, nadie levanta la mano, y si pierdes a alguien no te enteras. Cinco reglas que compensan eso:

- 1 • Narra antes de escribir, no después.** Di qué vas a hacer, luego hazlo. Si escribes primero y explicas después, la audiencia pasa el comando entero leyendo en vez de escuchando.
- 2 • Escribe despacio y en voz alta.** En una transmisión el retraso de video es de uno a tres segundos. Lo que a ti te parece un ritmo normal, del otro lado se ve como un salto.
- 3 • Cuando algo funcione, para dos segundos.** El silencio marca el momento. Si sigues hablando encima del resultado, nadie registra que acaba de pasar algo.
- 4 • Anuncia los tiempos muertos.** Mientras Claude responde hay de cinco a veinte segundos de nada. Llénalos con contexto, nunca con silencio: en transmisión el silencio se lee como falla técnica.

5 • Si una demo falla, córtala a los 45 segundos. No hay premio por arreglarla en vivo, y el costo de intentarlo es perder a la mitad de la audiencia. Pasa al respaldo y sigue.

Tabla de control

#	DEMO	MIN	RIESGO	RESPALDO
1	La skill en tres minutos	8	Medio	R1-apertura.md
2	Anatomía de un SKILL.md	7	Bajo	No necesita
3	La descripción, A/B en vivo	10	Alto	R3-descripcion.md
4	El destilador	12	Alto	R4-destilador.md
5	Multiarchivo y script	7	Medio	R5-multiarchivo.md
6	La skill que no funciona	8	Bajo	R6-diagnostico.md

Las demos 3 y 4 son las de mayor riesgo porque dependen de que el modelo responda algo concreto. Son también las dos que más enseñan. Ensáyalas el doble que las demás.

DEMO 1 • La skill en tres minutos

Minuto 0:02 • Duración 6 min • Riesgo medio

Objetivo

Que la audiencia vea una skill funcionando **antes** de saber qué es. La promesa se demuestra, no se enuncia.

Estado inicial

- Terminal abierta en `~/demo-webinar`
- `revisando-cambios` instalada
- 12 líneas de diff sin commitear
- Claude Code **cerrado** (arrancas en vivo)

Entrada

Erick abre el webinar y cierra su apertura con:

"...primero se los voy a mostrar, con código real, y después vemos qué pasó. Adelante."

Esa es tu señal. **No te presentes con tu currículum.** Si quieres decir quién eres, una línea y nada más: "soy <nombre>, y voy a llevar la parte práctica".

Guion

Paso 1 • El encuadre, sin diapositiva (40 s)

Antes de explicar nada les quiero mostrar algo. Esto que van a ver toma tres minutos de configurar, una sola vez, y después funciona solo.

Comparte la terminal. No un deck.

Paso 2 • El problema, en su propio código (50 s)

```
cd ~/demo-webinar
git diff HEAD
```

Mientras se muestra el diff:

Esto es un cambio normal de un martes cualquiera. Agregué cupones de descuento a un calculador de facturas. Doce líneas. Antes de subirlo quiero que alguien lo revise, y no hay nadie disponible.

Señala con el cursor, sin leer el código línea por línea.

Paso 3 · La pregunta normal (30 s)

```
claude
```

Espera a que abra. Luego escribe **despacio**:

```
¿qué cambié y qué debería revisar antes de subirlo?
```

Fíjense en lo que acabo de escribir. No es un comando. No escribí ninguna diagonal. Es la pregunta que le haría a un compañero.

Paso 4 · El momento (2 min)

Mientras responde, **no te calles**:

Lo que está pasando ahora es que Claude leyó la descripción de las skills que tengo instaladas, comparó con lo que le pedí, y encontró una que coincide. Está corriendo `git diff` por su cuenta porque la skill se lo indica.

Cuando termine, **para dos segundos**. Luego:

No le dije qué revisar. No le di el criterio. No le pegué el diff. Todo eso estaba escrito en un archivo, una sola vez, hace meses.

Paso 5 · Enseña el archivo (1 min)

```
/exit  
cat ~/.claude/skills/revisando-cambios/SKILL.md
```

Esto es todo. Veinte líneas. Un archivo de texto en una carpeta. Eso es una skill.

Paso 6 · Salida (30 s)

En los próximos noventa minutos vamos a ver qué tiene adentro ese archivo, por qué se disparó solo, y cómo escriben ustedes el suyo.

Frase de cierre:

Todo eso estaba escrito en un archivo, una sola vez.

Te callas. Erick arranca desde ahí con el bloque del mecanismo.

Señal de éxito

La skill se dispara **sin** que escribas `/`, y el resultado menciona al menos el `KeyError` o el `1.16` hardcodeado.

Cuando falla

QUÉ PASA	QUÉ HACES
No se dispara sola	Invócala con <code>/revisando-cambios</code> y di: <i>"aquí la estoy llamando yo; en un momento vemos por qué a veces no se dispara sola — es el tema de la demo 3"</i> . Convierte el fallo en anticipo
Tarda más de 40 s	Sigue narrando. Si pasa de 60 s, corta y pasa al respaldo
La respuesta es vaga o genérica	Léela igual y di: <i>"aquí no encontró gran cosa; en la demo 3 vamos a ver que eso casi siempre es la descripción"</i>
Error de sesión o red	Respaldo <code>R1-apertura.md</code> , sin intentar arreglar

DEMO 2 · Anatomía de un SKILL.md

Minuto 0:15 · Duración 7 min · Riesgo bajo

Objetivo

Que quede clarísima la distinción **descripción siempre en contexto** / **cuerpo solo al invocar**. Es el concepto del que depende todo lo demás.

Estado inicial

Terminal con el `SKILL.md` de la demo 1 ya visible, o listo para abrirse.

Entrada

Erick cierra el bloque *Por qué existen las skills* con:

"La descripción siempre está en contexto. El cuerpo, no. Ese es todo el truco... y se ve mejor en el archivo. Adelante."

Guion

Paso 1 · Las dos mitades (2 min)

```
cat ~/.claude/skills/revisando-cambios/SKILL.md
```

Señala físicamente con el cursor la parte de arriba:

Arriba, entre los tres guiones, están los metadatos. Esta parte está **siempre** en el contexto de Claude. En todas sus conversaciones, ahora mismo, aunque no la usen.

Ahora la parte de abajo:

Abajo está el cuerpo. Esta parte **solo** entra cuando la skill se usa. Puede tener cuatrocientas líneas y no cuesta nada mientras no se invoque.

Paso 2 · Por qué eso importa (2 min)

Podrían tener cincuenta skills instaladas. Lo que está permanentemente ocupando contexto son cincuenta descripciones de dos líneas. Los cincuenta cuerpos no están ahí hasta que hacen falta. Esa es la diferencia con meter todo en un CLAUDE.md gigante, y es la razón por la que el sistema existe.

Paso 3 · El listado en vivo (2 min)

claude

/skills

Aquí está lo que Claude ve permanentemente: los nombres y las descripciones. Nada más. Los cuerpos están en disco, esperando.

Paso 4 · El detalle que casi nadie sabe (1 min)

Una cosa más, y es la que cambia cómo escriben el cuerpo: cuando una skill se invoca, su contenido **se queda** en la conversación en los turnos siguientes. No es un préstamo momentáneo. Eso significa que cada línea del cuerpo es un costo recurrente, no un costo único. Y significa que se escriben instrucciones permanentes, no pasos de un solo uso.

Salida

Frase de cierre:

La descripción siempre está en contexto. El cuerpo, no.

Erick retoma con *Dónde viven y quién gana*.

Señal de éxito

/skills lista revisando-cambios y revisando-readme con sus descripciones visibles.

Cuando falla

QUÉ PASA	QUÉ HACES
/skills no existe en tu versión	Usa ¿qué skills tienes disponibles? — funciona igual
La lista sale muy larga	Es buena señal, no mala. Di: "aquí hay skills de varios orígenes; en un momento vemos de dónde sale cada una"

DEMO 3 · La descripción, A/B en vivo

Minuto 0:27 · Duración 10 min · Riesgo ALTO

Objetivo

Demostrar que **la descripción es lo único que decide** si una skill existe en la práctica. Es la demo que más cambia el comportamiento de la audiencia.

Por qué es de alto riesgo

Depende de que el modelo **no** dispare una skill con descripción mala y **sí** dispare la corregida. Es probabilístico. Ensáyala tres veces mínimo.

Decisión de diseño: si en el ensayo la versión mala se dispara de todos modos, **no fuerces la demo**. Cambia a la variante narrada del respaldo `R3-descripcion.md`, que enseña lo mismo sin depender del azar.

Estado inicial

Repositorio limpio, sin skills adicionales.

Entrada

Erick cierra *Dónde viven* y *quién gana* con el detalle del nombre de directorio y te entrega:

"...y ahora la parte donde casi todos fallamos la primera vez. Adelante."

Guion

Paso 1 · Escribe la versión mala, en vivo (2 min)

```
mkdir -p ~/.claude/skills/ayudante-docs
```

Abre el editor en pantalla y escribe **despacio**, narrando:

```
---
description: Te puedo ayudar con tus documentos y archivos.
---

## Instrucciones

Revisa el README y dime qué secciones le faltan.
```

Esta descripción parece perfectamente razonable. Yo escribí muchas así.

Paso 2 · Pruébala (2 min)

```
claude
```

```
revisa mi README y dime qué le falta
```

Lo esperado: no se dispara, o responde genéricamente sin usar la skill.

No pasó nada. La skill está instalada, el archivo existe, y no se activó.

Paso 3 · El diagnóstico, por partes (3 min)

Sal y abre el archivo otra vez. Corrige **una cosa a la vez**, diciendo cada una en voz alta:

```
description: Revisa un README y señala las secciones que faltan.
```

Primero: tercera persona. "Te puedo ayudar" está escrito como si Claude le hablara al usuario, pero esta descripción se inyecta en la indicación del sistema. El cambio de persona rompe el descubrimiento.

```
description: Revisa un README y señala las secciones que faltan.  
Úsala cuando el usuario mencione README, documentación o formato.
```

Segundo: faltaba el **cuándo**. La fórmula completa es qué hace, más cuándo usarla. Sin la segunda mitad, Claude sabe qué hace pero no en qué momento aplicarla.

Y tercero, el más sutil: las palabras. "Documentos y archivos" no es lo que ustedes escriben. Ustedes escriben "revisa este README".

Paso 4 · Pruébala otra vez (2 min)

```
claude
```

```
revisa mi README y dime qué le falta
```

Ahora sí se dispara. Para dos segundos.

Paso 5 · La conclusión (1 min)

Mismo archivo. Mismo cuerpo. Mismo todo, excepto una línea. Y la diferencia entre una skill que no existe y una que funciona. Si se llevan una sola cosa de este webinar, que sea esta: cuando una skill no se dispara, el problema casi nunca está en el cuerpo. Está en la descripción.

Salida

Frase de cierre:

Si no se dispara, el problema está en la descripción.

Erick retoma con *Las cuatro reglas* y consolida lo que acabas de demostrar. **No las enumeres tú:** son de su bloque, y si las dices aquí él se queda sin contenido.

Señal de éxito

La versión mala **no** se dispara; la corregida **sí**.

Cuando falla

QUÉ PASA	QUÉ HACES
La versión mala Sí se dispara	Es el fallo más probable. Di: <i>"aquí sí se disparó, y eso también pasa: el modelo a veces adivina. Pero adivinar no es un mecanismo confiable. Veamos por qué"</i> y sigue con las correcciones. El punto se sostiene igual
La corregida no se dispara	Usa exactamente las palabras de la descripción: <code>revisa mi README y dime qué secciones le faltan</code> . Luego explica que la coincidencia de palabras es literalmente el mecanismo
Te equivocas al teclear	Déjalo. Corregir un typo en vivo humaniza; borrar y reescribir todo, no

DEMO 4 · El destilador

Minuto 0:47 · Duración 12 min · Riesgo ALTO

Objetivo

La demo más valiosa del webinar. Demuestra el mecanismo que la audiencia va a usar mañana: convertir una conversación que ya funcionó en una skill.

Por qué es de alto riesgo

Es la más larga y depende de que el modelo produzca un `SKILL.md` decente en vivo. Además es la que más tienta a improvisar.

Estado inicial

Repositorio de demostración, con el diff sin commitear.

Entrada

Erick cierra la primera tanda de preguntas del chat y te entrega:

"...y esa última pregunta la contesta la demostración que sigue, que es la que yo usaría mañana. Adelante."

Aquí está el punto de control del webinar. Si esta entrada llega después del minuto 50, Erick va a recortar de las demos 5 y 6, no de esta.

Guion

Paso 1 · Resuelve algo a mano, sin skill (3 min)

```
claude
```

```
mira el diff de este repo y redáctame el mensaje de commit.  
Usamos commits convencionales: tipo entre paréntesis el alcance,  
resumen en imperativo y menos de 72 caracteres. El cuerpo explica  
por qué, no qué.
```

Mientras responde:

Fíjense en todo lo que tuve que escribir. El formato, la convención, el límite de caracteres, qué va en el cuerpo. Eso es contexto que yo puse, y que voy a tener que volver a poner mañana, y pasado.

Paso 2 · Pide la destilación (3 min)

Ahora, en lugar de copiarme esto a un archivo de notas, le voy a pedir que lo convierta en una skill. Y no se lo voy a pedir en general: le voy a dar un procedimiento.

Pega el **prompt destilador corto** (lo tienes en la tarjeta de cabina):

```
Destila esta conversación en un SKILL.md de Claude Code.  
  
Nombre en gerundio. Descripción en tercera persona con la fórmula  
[qué hace] + "Úsala cuando..." usando las palabras que yo usé de verdad  
en esta conversación, no lenguaje de manual. En el cuerpo pon solo el  
contexto que tuve que darte y que no sabías; quita todo lo que ya sabías.  
Menos de 60 líneas. Termina con un comentario del grado de libertad.  
  
Dame el archivo y el mkdir. Nada más.
```

Mientras genera:

Le estoy pidiendo tres cosas específicas. Que use **mis** palabras, no las del manual. Que quite lo que ya sabía, porque eso solo ocuparía contexto. Y que me diga qué tan frágil es la tarea.

Paso 3 · Léelo en voz alta (2 min)

Cuando aparezca, **no lo instales todavía**. Léelo señalando:

Miren la descripción. Dice "commits convencionales" y "mensaje de commit", que son exactamente las palabras que yo escribí. No dice "gestión de control de versiones". Y miren lo que **no** está: no me explica qué es un commit. Eso ya lo sabe.

Paso 4 · Instálala y pruébala en limpio (3 min)

```
/exit
```

Crea la carpeta y pega el archivo. Luego, **sesión nueva**:

```
claude
```

```
dame un mensaje de commit para esto
```

Sesión nueva. No le expliqué el formato. No le dije la convención. Y ahí está.

Paso 5 · El cierre del ciclo (1 min)

A esto le llamamos el ciclo Claude A / Claude B. A es donde resuelves y destilas. B es una sesión limpia donde pruebas. Lo que observan en B regresa a A. Nadie escribe un SKILL.md bueno de memoria y a la primera. Este ciclo es el trabajo real, y está en el kit que se llevan, con el prompt completo en su versión larga.

Salida

Frase de cierre:

Nadie escribe un SKILL.md bueno de memoria y a la primera.

Erick retoma con *Grados de libertad*.

Señal de éxito

La skill destilada se dispara en sesión nueva y produce un mensaje de commit con el formato que pediste.

Cuando falla

QUÉ PASA	QUÉ HACES
El SKILL.md generado es muy largo	Úsalo a tu favor: pide <code>recórtalo a la mitad, quita lo que ya sabes</code> en vivo. Es exactamente lo que enseña el paso 4 del manual
La descripción sale en lenguaje de manual	Ídem: <code>reescribe la descripción con las palabras que yo usé, no las del manual</code> . La corrección en vivo enseña más que el acierto
No se dispara en la sesión nueva	Es la demo 3 otra vez, y es buena noticia: " <i>vean, me pasó a mí en vivo. Volvemos a la descripción</i> ". Corrígela y reprueba
Se acaba el tiempo	Corta en el paso 3. Muestra la skill ya instalada del respaldo y salta al paso 5

DEMO 5 · Multiarchivo y script

Minuto 1:05 · Duración 7 min · Riesgo medio

Objetivo

Mostrar que una skill puede ejecutar código, y enseñar el patrón que evita la solicitud de permiso.

Estado inicial

revisando-readme instalada, con su reference.md y su script.

Entrada

Erick cierra Grados de libertad con el error típico y te entrega:

"...y una skill de libertad media casi siempre tiene un script adentro. Adelante."

Guion

Paso 1 · La estructura (1 min)

```
tree ~/.claude/skills/revisando-readme 2>/dev/null || find ~/.claude/skills/revisando-readme
```

Tres piezas. El SKILL.md con el procedimiento. Un reference.md con el detalle, que se carga solo si hace falta. Y un script, que se **ejecuta**, no se lee.

Paso 2 · El patrón anti-permiso (2 min)

```
head -8 ~/.claude/skills/revisando-readme/SKILL.md
```

Señala la línea de `allowed-tools` :

Aquí está el truco que vale su peso en oro. La ruta del script aparece en dos lugares: en el cuerpo, donde le digo que lo ejecute, y aquí en `allowed-tools`, con la misma variable. Como coinciden exactamente, el script corre **sin preguntarme permiso**. Sin este patrón, cada ejecución me interrumpe.

Paso 3 · Corre la skill (3 min)

```
claude
```

¿mi README está completo?

Mientras corre:

Está ejecutando el script, leyendo el resultado, y solo después va a consultar el reference.md si lo necesita. Eso es revelación progresiva: el detalle se carga cuando hace falta, no antes.

Paso 4 · Las tres reglas (1 min)

Tres reglas donde todo el mundo se equivoca. Las referencias van a un solo nivel, nunca en cadena. Los scripts se ejecutan, no se leen. Y las rutas usan la variable, nunca rutas absolutas de tu máquina.

Salida

Frase de cierre:

Los scripts se ejecutan, no se leen.

Esta es la única demo que **no** devuelve la pantalla: sigue la demo 6, que también es tuya. Enlaza directo:

Y con eso ya podemos ver lo contrario: una skill que tiene todo esto y no funciona.

Señal de éxito

La skill ejecuta el script **sin** pedir permiso, y el reporte menciona secciones faltantes del README.

Cuando falla

QUÉ PASA	QUÉ HACES
Pide permiso	Acéptalo y di: "aquí me pidió permiso, y eso significa que la regla de allowed-tools no coincide con el comando. Es el error más común de este patrón". Enseña más que el éxito
El script no es ejecutable	<code>chmod +x</code> en vivo, con una línea: "detalle de permisos de archivo, nada que ver con skills"
No encuentra reference.md	Revisa que uses la variable y no una ruta relativa. Si falla, sigue: el punto principal ya se demostró

DEMO 6 · La skill que no funciona

Minuto 1:12 · Duración 8 min · Riesgo bajo

Objetivo

Dejar instalada la capacidad de diagnóstico, y cerrar con el punto de que las fallas más caras son las que ninguna herramienta señala.

Estado inicial

`doc-helper` no instalada. La instalas en vivo.

Entrada

Vienes de la demo 5 sin devolver la pantalla. No hay entrada de Erick.

Guion

Paso 1 · Instala la saboteada (1 min)

```
cp -r <ruta-del-kit>/ambiente-demo/skills-demo/doc-helper ~/.claude/skills/  
cat ~/.claude/skills/doc-helper/SKILL.md
```

Esta skill tiene tres fallas. Está diseñada para no activarse nunca. A ver si las ven antes que yo las diga.

Deja el archivo en pantalla **quince segundos en silencio**. En un webinar, ese silencio hace que la gente lea de verdad.

Paso 2 · Las tres fallas (4 min)

Falla 1 · el frontmatter

`Name` con mayúscula no es un campo válido. El campo es `name`, minúscula. Y el valor tiene espacios y mayúsculas, que tampoco se permiten. ¿Qué pasa? La skill carga **sin metadatos**. Responde si la invocan con diagonal, pero nunca se dispara sola.

Falla 2 · la descripción

"Te puedo ayudar con tus documentos y archivos". Segunda persona, sin el cuándo, con palabras que nadie teclea. Las tres cosas de la demo 3. Esta es la que se escapa. Parece una descripción normal.

Falla 3 · las rutas

`scripts\formato.md` con barra invertida, que no resuelve. Le pide leer un script en vez de ejecutarlo. Y el `referen-
ce.md` apunta a dos archivos más: una cadena de dos niveles, que funciona a veces y falla otras.

Paso 3 · Las herramientas (2 min)

```
claude
```

```
/doctor
```

`/doctor` para la instalación. `/skills` para ver qué cargó y de dónde. `/context` para ver qué está ocupando la ventana. Y `claude --debug` para los errores de parseo. Y hay una más, que encuentra frontmatter roto en un directorio entero:

```
claude plugin validate ~/.claude/skills
```

Paso 4 · El cierre (1 min)

Dos de las tres fallas eran de sintaxis, y las herramientas se las señalan. La tercera era una descripción que se veía perfectamente bien. Esa es la que van a tener en producción. Y ninguna herramienta se las va a marcar.

Salida

Frase de cierre:

Las descripciones malas no las marca ninguna herramienta.

Dejas de compartir pantalla. Erick cierra el webinar con el kit y el compromiso. **Tu parte terminó aquí**, salvo las preguntas técnicas de la última tanda.

Señal de éxito

La audiencia identifica al menos una falla en el chat antes de que la digas.

Cuando falla

QUÉ PASA	QUÉ HACES
<code>claude plugin validate</code> no existe	Requiere v2.1.233+. Di la versión y sigue con <code>--debug</code>
Nadie responde en el chat	Normal en webinar. Di: <i>"la primera está en la primera línea"</i> y sigue
Alguien encuentra una cuarta	Celébralo y pregunta cuál. Es el mejor momento posible de la demo

Apéndice • Si van retrasados

El recorte lo decide Erick, no tú. Tú lo ejecutas. Esta tabla es para que sepas qué esperar cuando te lo pida.

RETRASO	QUÉ SE RECORTA
5 min	La tanda 1 de preguntas, de 5 a 2 min. No te afecta
10 min	Demo 5 completa. El patrón anti-permiso se menciona sin demostrar
15 min	Además, demo 6 a 4 min: solo la falla 2 y las herramientas
20 min	Además, demo 2 a 3 min: solo las dos mitades del archivo

Nunca se recorta: la demo 1, la demo 4, ni el cierre con el kit.

La demo 1 es la promesa. La 4 es lo que van a usar. El cierre es lo único que convierte a un asistente en alguien que aplica algo.

Cada respaldo trae, al final, su propia sección de **"si vas retrasado"** con el recorte ya decidido. Úsala en vez de improvisar qué quitar.

Apéndice · Frases de cierre de cada demo

Memoriza estas seis. Son lo único que la audiencia debe poder repetir al día siguiente.

- 1. Demo 1** — "Todo eso estaba escrito en un archivo, una sola vez."
- 2. Demo 2** — "La descripción siempre está en contexto. El cuerpo, no."
- 3. Demo 3** — "Si no se dispara, el problema está en la descripción."
- 4. Demo 4** — "Nadie escribe un SKILL.md bueno de memoria y a la primera."
- 5. Demo 5** — "Los scripts se ejecutan, no se leen."
- 6. Demo 6** — "Las descripciones malas no las marca ninguna herramienta."